DISEÑO EN HARDWARE DEL ALGORITMO HAMMING QUACI-CYLIC (HQC) PARA ENCRIPTACIÓN POSTCUÁNTICA

Camilo Home Buriticá



Escuela de Ingeniería Eléctrica y Electrónica

Facultad de Ingeniería

Santiago de Cali

Agosto 2025

DISEÑO EN HARDWARE DEL ALGORITMO HAMMING QUACI-CYLIC (HQC) PARA ENCRIPTACIÓN POSTCUÁNTICA

Camilo Home Buriticá

Documento presentado a
Programa Académico de Ingeniería Electrónica
como propuesta de trabajo de grado

Director: Jaime Velasco Medina, Ph.D., M.Sc., Ing.
Grupo de Investigación en Bionanoelectrónica (BIONANO)

Codirectora: Claudia Patricia Renteria, Ph.D., M.Sc., Ing.
Grupo de Investigación en Bionanoelectrónica (BIONANO)



Escuela de Ingeniería Eléctrica y Electrónica
Facultad de Ingeniería
Santiago de Cali
Agosto 2025

TABLA DE CONTENIDO

Pag.

FICHA RESUMEN **1**

INTRODUCCIÓN **2**

1. PLANTEAMIENTO DEL PROBLEMA **3**

2. ANTECEDENTES **5**

2.1. ANTECEDENTES INTERNACIONALES 5

3. OBJETIVOS **6**

3.1. OBJETIVO GENERAL 6
3.2. OBJETIVOS ESPECÍFICOS 7

4. JUSTIFICACIÓN **7**

5. MARCO TEÓRICO **8**

5.1. La computación cuántica y su impacto en la criptografía 8
5.2. Criptografía postcuántica 8
5.3. El algoritmo Hamming Quasi-Cyclic (HQC) 9
5.4. Estado del Arte en Implementación de HQC en Hardware 14

6. METODOLOGÍA **15**

7. CRONOGRAMA **17**

8. PRESUPUESTO **18**

REFERENCIAS **19**

FICHA RESUMEN

Título:	DISEÑO EN HARDWARE DEL ALGORITMO HAMMING QUASI-CYLIC (HQC) PARA ENCRIPTACIÓN POSTCUÁNTICA
Palabras clave:	Criptografía postcuántica, computación cuántica, diseño en FPGA, encapsulación de clave, ciberseguridad.
Resumen:	Este proyecto presenta el diseño en hardware del algoritmo Hamming Quasi-Cyclic, un mecanismo de encapsulación de clave resistente a ataques cuánticos seleccionado por el NIST como quinto algoritmo dentro del proceso de estandarización de criptografía post-cuántica. Debido al avance acelerado de la computación cuántica y su potencial para vulnerar sistemas criptográficos clásicos, se propone evaluar la viabilidad de HQC en plataformas FPGA, optimizando su rendimiento en velocidad, uso de recursos y consumo energético. Para ello, se desarrolla una arquitectura digital en VHDL, validada mediante simulaciones y comparaciones con diseños existentes, aportando al avance de soluciones seguras y eficientes frente a futuras amenazas en ciberseguridad, sentando bases para futuras aplicaciones en sistemas embebidos y redes seguras.
Área temática:	Implementación de Algoritmos Criptográficos en Hardware
Modalidad de trabajo de grado:	☒ Profesional ☐ Revisión crítica ☐ Investigación e innovación ☐ En la industria/empresa ☐ Creación de empresa ☐ Práctico social
Duración del trabajo de grado:	10 Meses
Entidades participantes:	Universidad del Valle
Costo presupuestado del trabajo:	Aportes en especie: COP$ 18'672.000 Aportes en efectivo: COP$ 0 Costo global: COP$ 18'672.000
Grupos de investigación:	☐ Alta tensión (GRALTA) ☐ Arquitecturas digitales y microelectrónica (GADYM) ☒ Bionanoelectrónica (BIONANO) ☐ Control industrial (GICI) ☐ Conversión de energía (CONVERGIA) ☐ Percepción y sistemas inteligentes (PSI) ☐ Sistemas de telecomunicaciones (SISTEL-UV) ☐ Sistema de hidroeléctricos de generación (SIIG) ☐ Otro:
Estudiante	
Nombre:	Camilo Home Buriticá **Código:** 202043403
Correo electrónico:	camilo.home@correounivalle.edu.co **Tel.:** (57) 3226928217
Director	
Nombre:	Jaime Velasco Medina **Tel.:** (57) 6023212100 ext. 3283/7390
Correo electrónico:	jaime.velasco@correounivalle.edu.co
Institución:	Universidad del Valle - Bionanoelectrónica (BIONANO)

INTRODUCCIÓN

El desarrollo de la computación cuántica esta avanzando rápidamente y se ha convertido en uno de los mayores desafíos para los métodos de criptográfica convencionales comprometiendo la seguridad de toda la información. A diferencia de la computación clásica, los computadores cuánticos permiten ejecutar algoritmos que reducen drásticamente la complejidad de ciertos problemas matemáticos sobre los cuales se fundamentan los principales algoritmos criptográficos actuales, como RSA y ECC. En particular, el algoritmo de Shor permite la factorización de enteros y la resolución del logaritmo discreto en tiempo polinomial, anulando directamente la seguridad de estos algoritmos [1]. Esto ha motivado a la comunidad científica y en especial al Instituto Nacional de Estándares y Tecnología (NIST) a iniciar un proceso de estandarización de algoritmos resistentes a ataques cuánticos, bajo el campo emergente de la criptografía post-cuántica.

En este contexto, el algoritmo Hamming Quasi-Cyclic (HQC) ha sido seleccionado por el NIST en su cuarta ronda de evaluación, como el quinto sistema criptográfico en ser estandarizado, específicamente como mecanismo para encapsulación de claves (KEM) en reemplazo de ML-KEM [2]. HQC es un algoritmo de clave pública estructurado en códigos de corrección de errores, basándose en la dificultad del problema de decodificación de síndromes en códigos quasi-cíclicos. A diferencia de los otros algoritmos ya seleccionados, HQC ofrece una estructura alternativa y eficiente, lo cual lo convierte en una poderosa opción tanto en términos de seguridad como de implementación.

Sin embargo, el despliegue real de HQC en sistemas de transmisión de información dependerá en gran medida de si es viable o no su ejecución en hardware. La implementación eficiente de este algoritmo en plataformas reconfigurables como FPGAs, es clave para validar su integración en sistemas embebidos, dispositivos de red y protocolos de comunicación. En este orden de ideas, este proyecto tiene como objetivo desarrollar una arquitectura en hardware para HQC, siguiendo las especificaciones más recientes publicadas por el NIST. El diseño se enfocará en optimizar el rendimiento en términos de velocidad, consumo de recursos lógicos y eficiencia energética, utilizando descripciones RTL en VHDL, herramientas de síntesis como Quartus Prime y una plataforma FPGA Cyclone IV (DE2 - 115) para validación funcional y evaluación de rendimiento.

Este trabajo no solo busca validar la implementación funcional del algoritmo HQC en hardware, sino también aportar a la investigación de soluciones criptográficas que logren

enfrentar las amenazas de la era cuántica.

1. PLANTEAMIENTO DEL PROBLEMA

El avance de las tecnologías de la información y la comunicación (TIC) ha transformado completamente la manera en la que los seres humanos nos comunicamos e intercambiamos información. Desde la unificación de las telecomunicaciones y las redes informáticas hasta el almacenamiento de datos y la implementación del internet de las cosas (IoT), la transmisión de información se ha convertido en una actividad fundamental en nuestro mundo moderno. A diario se transmiten grandes cantidades de datos, de los cuales muchos son de carácter sensible o privado. Con el fin de garantizar la confidencialidad, integridad y autenticidad de la información se utiliza la criptografía, la cual se encarga de desarrollar métodos de cifrado para modificar la representación de ciertos mensajes haciéndolos incomprensibles a receptores que no tengan la autorización para acceder a su contenido [3].

Los algoritmos criptográficos basados en problemas matemáticos complejos, como RSA (uso de clave publica), AES (cifrado simétrico) y ECC (criptografía de curva elíptica), han brindado un alto nivel de seguridad durante décadas. Sin embargo, el acelerado desarrollo de la computación cuántica a lo largo de los últimos años ha demostrado que algoritmos como el de Shor, puede romper RSA y ECC, y el algoritmo de Grover vulnera la seguridad del algoritmo AES a la mitad, limitándolo a ser usado en la era de la computación cuántica solo con una clave de 256-bit [4]. Esto nos deja como resultado un panorama donde la información encriptada bajo estos algoritmos está totalmente expuesta, lo que pondría en riesgo la seguridad de aplicaciones bancarias, redes gubernamentales, infraestructura crítica, etc.

Adicionalmente, otro de los principales riesgos emergentes es la práctica conocida como Harvest Now, Decrypt Later (HNDL), que en español significa “recolectar ahora, desencriptar después”, la cual es una estrategia que se basa en la adquisición y almacenamiento a largo plazo de información actualmente cifrada con el objetivo de descifrarla en un futuro cuando se disponga de la suficiente potencia de cómputo cuántico [5]. Se estima que hay un alto número de actores maliciosos, especialmente atacantes de entes gubernamentales, que estarían robando datos cifrados muy sensibles con fines políticos y sociales.

La aparición de este tipo de estrategias ha impulsado a la comunidad criptográfica a desarrollar con urgencia nuevos algoritmos de encriptación, lo cual ha dado origen a un nuevo estándar que reemplazará la criptografía de clave pública actual con la criptografía post-cuántica, termino con el que ha sido denominada la criptografía resistente a los ataques cuánticos. Este Proceso de estandarización se ha llevado a cabo por el Instituto Nacional de Estándares y Tecnología (NIST) con el fin de seleccionar los algoritmos que reemplacen los algoritmos tradicionales de seguridad para firma digital y cifrado e intercambio de claves [6].

Dentro de estos esfuerzos, el algoritmo Hamming Quasi-Cyclic (HQC) fue seleccionado en la categoría Key Encapsulation Mechanism (KEM) como el quinto y último algoritmo en ser estandarizado hasta la fecha. En febrero de 2025 se publicó su última especificación, la cual introduce optimizaciones en eficiencia y seguridad [7]. Este algoritmo servirá como respaldo de ML-KEM, uno de los algoritmos actuales para criptografía de clave pública dado que el HQC se basa en una matemática diferente a la de ML-KEM, lo cual podría ser importante en caso de que se descubriesen nuevas vulnerabilidades en dicho algoritmo. NIST planea publicar un borrador de estándar que incorpore el algoritmo HQC en el próximo año, y se espera que el estándar final esté listo para 2027 [8].

Dado que la adopción de HQC en sistemas reales dependerá en gran medida de su viabilidad en entornos de hardware se plantea la siguiente pregunta problematizadora:

¿Cómo desarrollar una implementación eficiente en hardware del algoritmo HQC que optimice el rendimiento en términos de velocidad, consumo de recursos y consumo de potencia, superando las limitaciones de implementaciones previas basadas en síntesis de alto nivel (HLS)?

Para responder esta pregunta, este trabajo se centrará en la implementación en hardware de HQC, obedeciendo a las especificaciones más recientes del NIST (febrero de 2025) [7], con el objetivo de analizar su desempeño y explorar posibles optimizaciones para su implementación en sistemas embebidos y redes de transmisión de datos de alta seguridad.

2. ANTECEDENTES

En este apartado se presentan trabajos que proponen avances en la implementación del algoritmo de encapsulación de clave Hamming Quasi-Cyclic (HQC), con el objetivo de optimizar su ejecución en hardware y hacerlo viable para sistemas embebidos, FPGA y otros entornos de computación segura.

Entre el contenido investigado referente al anteproyecto en cuestión, resultaron relevantes cuatro documentos en los que se proponen distintas arquitecturas de hardware eficientes para HQC. La búsqueda se enfocó en trabajos publicados a partir del año 2020, considerando el reciente proceso de estandarización del NIST y la evolución acelerada de las técnicas de diseño RTL.

Cabe mencionar que, hasta la fecha, no se han identificado antecedentes documentados en el ámbito nacional que aborden la implementación en hardware reconfigurable o en procesadores dedicados del algoritmo criptográfico postcuántico HQC. Esto evidencia una oportunidad relevante para contribuir al desarrollo local en el área de ciberseguridad postcuántica y su implementación en hardware.

2.1 ANTECEDENTES INTERNACIONALES

"*A High Efficiency Hardware Design for the Post-Quantum KEM HQC*" [9]. Presentan un diseño a nivel de transferencia de registro (RTL) altamente optimizado del algoritmo HQC KEM, plenamente compatible con la versión de especificación publicada en abril de 2023. Su propuesta mejora el rendimiento de los módulos más costosos —multiplicadores polinómicos y decodificadores Reed-Solomon/Muller, logrando aceleraciones de hasta 10.6× y reducciones importantes en el producto área-tiempo en comparación con las implementaciones previas en FPGA Artix-7.

"*An Efficient Hardware Design for Fast Implementation of HQC*" [10]. En este documento se propone una arquitectura de hardware que reduce significativamente la latencia en operaciones clave como la multiplicación polinómica y el decodificado. Emplean un decodificador Reed-Solomon basado en el algoritmo ePIBM y un decodificador Reed-Muller optimizado con FHT, obteniendo mejoras del 25 % en el área-tiempo y logrando una decapsulación en solo 0.38ms.

"*LTE: Lightweight and Time-Efficient Hardware Encoder for Post-Quantum Scheme HQC*"

[11]. En este trabajo se introduce una arquitectura ligera y eficiente en tiempo para el codificador de HQC. Su enfoque se centra en optimizar el flujo de datos entre los codificadores Reed-Solomon y Reed-Muller, así como en simplificar los multiplicadores en campos de Galois. Su implementación FPGA logra una reducción de hasta 35.2 % en el área-tiempo (ADP) para los tres niveles de seguridad HQC: 128, 192 y 256.

Finalmente, "*An Efficient and Unified RTL Accelerator Design for HQC-128, HQC-192, and HQC-256*" [12], presenta un diseño unificado y flexible para HQC que soporta dinámicamente los tres niveles de seguridad del estándar (HQC-128, HQC-192 y HQC-256), mediante un solo acelerador RTL configurable en tiempo de ejecución. Este diseño ofrece mejoras significativas en tiempo de ejecución, hasta 3× respecto a implementaciones especializadas, y logra reducir el uso de LUTs y FFs mediante la reutilización inteligente de componentes como los codificadores y decodificadores RS/RM, todo esto sin comprometer la compatibilidad con la especificación oficial del algoritmo.

Estos antecedentes validan que HQC no solo es robusto desde el punto de vista criptográfico, sino también eficiente y viable para ser implementado en hardware reconfigurable. Así, brindan una base sólida que respalda el desarrollo del criptoprocesador especializado en el mecanismo HQC con altos niveles de rendimiento y seguridad propuesto en este anteproyecto.

3. OBJETIVOS

3.1 OBJETIVO GENERAL

- Desarrollar una implementación en hardware del algoritmo de encriptación postcuántica HQC, siguiendo las especificaciones de NIST, buscando optimizar su rendimiento en velocidad, uso de recursos y consumo de potencia.

3.2 OBJETIVOS ESPECÍFICOS

1. Implementar una versión funcional del algoritmo HQC en alto nivel utilizando Python, que facilite comprensión de las principales operaciones.
2. Diseñar los bloques funcionales del algoritmo de encriptación HQC.
3. Desarrollar la arquitectura hardware del algoritmo HQC utilizando VHDL, teniendo en cuenta consumo de recursos, velocidad, y consumo de potencia.
4. Realizar una evaluación comparativa del desempeño con otras implementaciones hardware reportadas, verificando tiempos de ejecución, consumo de recursos y consumo de potencia

4. JUSTIFICACIÓN

Este proyecto se justifica por la necesidad urgente de desarrollar algoritmos de seguridad eficientes frente al avance de la computación cuántica. La reciente estandarización de HQC por parte del NIST refuerza su relevancia como algoritmo de respaldo en la futura infraestructura criptográfica global. Sin embargo, incluir este algoritmo en entornos reales requiere primero verificaciones de implementación en hardware, principalmente en dispositivos de bajo consumo y sistemas embebidos donde el software no es viable.

En la actualidad, la mayoria de las investigaciones sobre HQC están centradas en contextos internacionales, sin que se evidencien desarrollos aplicados en el ámbito nacional, particularmente en arquitecturas digitales reconfigurables como FPGA. Este vacío representa una oportunidad estratégica para fortalecer las capacidades de la región en ciberseguridad postcuántica dado que este proyecto aborda un tema de frontera. Su impacto no solo radica en la generación de conocimiento técnico local, sino también en sentar bases para futuras aplicaciones en dispositivos de red, IoT, defensa y otros sectores que requieren seguridad resistente a las amenazas cuánticas que están por venir.

5. MARCO TEÓRICO

5.1 La computación cuántica y su impacto en la criptografía

La computación cuántica representa un nuevo paradigma computacional basado en principios de la mecánica cuántica, como la superposición, el entrelazamiento y la interferencia. A diferencia de los bits clásicos que conocemos, que solo pueden tener dos estados, 0 o 1, los qubits (bits cuánticos) pueden representar ambos estados al mismo tiempo, lo que permite paralelismo masivo en ciertas operaciones computacionales.

Este poder computacional pone en riesgo la seguridad de los algoritmos de cifrado actuales. Algoritmos como el de Shor, propuesto en 1994 por el ingeniero en computación Peter Shor Williston, permite factorizar números enteros muy grandes en tiempo polinomial, este procedimiento seria totalmente ineficiente de llevar a cabo mediante lógica clásica, sin embargo, cuando se usan circuitos cuánticos el tiempo en encontrar la respuesta se vuelve significativamente inferior. Como todos los algoritmos de computación cuántica, el algoritmo de Shor es probabilístico, es decir que da la respuesta correcta con alta probabilidad y la probabilidad de fallo puede ser disminuida repitiendo el algoritmo. Muchos sistemas criptográficos de clave pública, tales como RSA y ECC, quedarían completamente obsoletos si el algoritmo de Shor es implementado alguna vez en una computadora cuántica práctica. Incluso con dispositivos cuánticos de tamaño limitado, ya se han demostrado ejecuciones de Shor sobre curvas elípticas y RSA de pocas bits. La amenaza se extiende también al enfoque de harvest now, decrypt later, donde datos cifrados hoy podrían ser almacenados y descifrados en el futuro una vez que los computadores cuánticos maduren [5].

5.2 Criptografía postcuántica

La criptografía postcuántica (PQC) agrupa algoritmos diseñados para resistir ataques tanto de computadores clásicos como cuánticos. Estos algoritmos no dependen de los problemas matemáticos clásicos, sino de problemas computacionales difíciles incluso para la lógica cuántica.

Los sistemas criptográficos postcuánticos existentes hasta el momento se pueden clasificar en 4 categorías:

- Basados en lattices (como Kyber, Dilithium).
- Basados en códigos (como Classic McEliece y HQC).
- Basados en funciones hash (como SPHINCS+).
- Basados en isogenias.

El NIST, en su proceso de estandarización PQC iniciado en 2016, seleccionó a HQC como uno de los cinco algoritmos finales recomendados, específicamente como mecanismo de encapsulacion de clave (KEM) de respaldo para ML-KEM. Esto refuerza la importancia de investigar su implementación y aplicabilidad [13].

Los cinco algoritmos actualmente estandarizados por el NIST son los siguientes:

1 CRYSTALS-KYBER (2022), algoritmo para encriptación general basado en lattices modulares.

2 CRYSTALS-DILITHIUM (2022), algoritmo para protección de firma digital basado en lattices modulares.

3 FALCON (2022), algoritmo para firma digital basado en lattices rápidos de fourier.

4 SPHINCS+ (2022), algoritmo para firma digital basado en funciones hash.

5 Hamming Quasi-Cyclic (HQC), algoritmo para encapsulación de clave basado en códigos de corrección de errores.

5.3 El algoritmo Hamming Quasi-Cyclic (HQC)

HQC es un algoritmo para encapsulación de claves (KEM) basado en códigos de corrección de errores cuasi-cíclicos, donde fortaleza del algoritmo radica en la dificultad del Síndrome Decoding Problem (SDP) para códigos binarios.

El esquema HQC–KEM se construye a partir de HQC–PKE, reutilizando sus funciones de generación de claves, cifrado y descifrado. Mientras HQC–PKE define un sistema de cifrado público, HQC–KEM lo transforma en un mecanismo de encapsulación de claves, añadiendo funciones de hash y derivación de claves (KDF) para lograr seguridad frente a ataques adaptativos (IND–CCA2).

5.3.1 HQC-PKE

5.3.1.1 Generación de claves

A partir de una semilla inicial se generan dos vectores secretos $\mathbf{x}$ y $\mathbf{y}$ de peso fijo, y un vector público $\mathbf{h}$. Con ellos se calcula $\mathbf{s} = \mathbf{x} + \mathbf{h} \cdot \mathbf{y}$, formando así la clave pública $(seed_{PKE.ek}, \mathbf{s})$ y la clave privada $seed_{PKE.dk}$.

Algoritmo HQC–PKE.KeyGen

Require: Semilla $seed_{PKE}$

Ensure: Clave pública ek_{PKE}, clave privada dk_{PKE}

▷ *Generar dos subsemillas aleatorias para cifrado y descifrado*

1: $(seed_{PKE.dk}, seed_{PKE.ek}) \leftarrow I(seed_{PKE})$

▷ *Generar los vectores* $\mathbf{x}$ *y* $\mathbf{y}$ *con peso Hamming fijo* ω

2: $ctx_{PKE.dk} \leftarrow \text{XOF.Init}(seed_{PKE.dk})$

3: $(ctx_{PKE.dk}, \mathbf{y}) \leftarrow \text{SampleFixedWeightVect\$}(ctx_{PKE.dk}, R_\omega)$

4: $(ctx_{PKE.dk}, \mathbf{x}) \leftarrow \text{SampleFixedWeightVect\$}(ctx_{PKE.dk}, R_\omega)$

5: $dk_{PKE} \leftarrow seed_{PKE.dk}$

▷ *Generar el parámetro público* $\mathbf{h}$

6: $ctx_{PKE.ek} \leftarrow \text{XOF.Init}(seed_{PKE.ek})$

7: $(ctx_{PKE.ek}, \mathbf{h}) \leftarrow \text{SampleVect}(ctx_{PKE.ek}, R)$

▷ *Calcular* $\mathbf{s} = \mathbf{x} + \mathbf{h} \cdot \mathbf{y}$

8: $\mathbf{s} \leftarrow \mathbf{x} + \mathbf{h} \cdot \mathbf{y}$

▷ *Formar las claves pública y privada*

9: $ek_{PKE} \leftarrow (seed_{PKE.ek}, \mathbf{s})$

10: **return** (ek_{PKE}, dk_{PKE})

5.3.1.2 Cifrado

El emisor usa la clave pública y una semilla aleatoria para generar los vectores $\mathbf{r}_1$, $\mathbf{r}_2$ y $\mathbf{e}$. Luego calcula $\mathbf{u}$ y $\mathbf{v}$, donde el mensaje se mezcla con un error controlado, produciendo el texto cifrado $\mathbf{c}_{PKE} = (\mathbf{u}, \mathbf{v})$.

Algoritmo HQC–PKE.Encrypt

Require: Clave pública ek_{PKE}, mensaje $\mathbf{m}$, aleatoriedad θ

Ensure: Texto cifrado $\mathbf{c}_{PKE}$

▷ *Interpretar la clave pública* ek_{PKE}

1: $seed_{PKE.ek} \leftarrow ek_{PKE}[0{:}|seed|]$

2: $ctx_{PKE.ek} \leftarrow \text{XOF.Init}(seed_{PKE.ek})$

3: $(ctx_{PKE.ek}, \mathbf{h}) \leftarrow \text{SampleVect}(ctx_{PKE.ek}, R)$

4: $\mathbf{s} \leftarrow ek_{PKE}[|seed|{:}|seed| + |\mathbf{s}|]$

▷ *Generar los vectores aleatorios* $\mathbf{r}_1$, $\mathbf{r}_2$ *y* $\mathbf{e}$ *con pesos fijos*

5: $ctx_\theta \leftarrow \text{XOF.Init}(\theta)$

6: $(ctx_\theta, \mathbf{r}_2) \leftarrow \text{SampleFixedWeightVect}(ctx_\theta, R_{\omega_r})$

7: $(ctx_\theta, \mathbf{e}) \leftarrow \text{SampleFixedWeightVect}(ctx_\theta, R_{\omega_e})$

8: $(ctx_\theta, \mathbf{r}_1) \leftarrow \text{SampleFixedWeightVect}(ctx_\theta, R_{\omega_r})$

▷ *Calcular* $\mathbf{u} = \mathbf{r}_1 + \mathbf{h} \cdot \mathbf{r}_2$

9: $\mathbf{u} \leftarrow \mathbf{r}_1 + \mathbf{h} \cdot \mathbf{r}_2$

▷ *Codificar el mensaje* $\mathbf{m}$ *y calcular* $\mathbf{v} = C.Encode(\mathbf{m}) + Truncate(\mathbf{s} \cdot \mathbf{r}_2 + \mathbf{e}, \ell)$

10: $\mathbf{v} \leftarrow C.\text{Encode}(\mathbf{m}) + \text{Truncate}(\mathbf{s} \cdot \mathbf{r}_2 + \mathbf{e}, \ell)$

▷ *Formar el texto cifrado* $\mathbf{c}_{PKE} = (\mathbf{u}, \mathbf{v})$

11: $\mathbf{c}_{PKE} \leftarrow (\mathbf{u}, \mathbf{v})$

12: **return** $\mathbf{c}_{PKE}$

5.3.1.3 Descifrado

El receptor reconstruye el vector secreto $\mathbf{y}$ a partir de su clave privada y usa $\mathbf{u}$ y $\mathbf{v}$ para eliminar el error y recuperar el mensaje original mediante el decodificador del código C.

Algoritmo HQC–PKE.Decrypt

Require: Clave privada dk_{PKE}, texto cifrado $\mathbf{c}_{PKE}$

Ensure: Mensaje $\mathbf{m}$

▷ *Interpretar la clave privada* dk_{PKE}

1: $seed_{PKE.dk} \leftarrow dk_{PKE}[0{:}|seed|]$

2: $ctx_{PKE.dk} \leftarrow \text{XOF.Init}(seed_{PKE.dk})$

3: $(ctx_{PKE.dk}, \mathbf{y}) \leftarrow \text{SampleFixedWeightVect\$}(ctx_{PKE.dk}, R_\omega)$

▷ *Interpretar el texto cifrado* $\mathbf{c}_{PKE}$

4: Analizar $\mathbf{c}_{PKE}$ como $(\mathbf{u}, \mathbf{v})$

▷ *Calcular* $\mathbf{v}' = \mathbf{v} - \mathit{Truncate}(\mathbf{u} \cdot \mathbf{y}, \ell)$

▷ *Decodificar* $\mathbf{v}'$ *usando el decodificador RMRS para recuperar* $\mathbf{m}$

5: $\mathbf{m} \leftarrow C.\text{Decode}(\mathbf{v} - \text{Truncate}(\mathbf{u} \cdot \mathbf{y}, \ell))$

▷ *Devolver el mensaje decodificado*

6: **return** $\mathbf{m}$

5.3.2 HQC-KEM

5.3.2.1 Generación de claves

Se generan vectores secretos y se calcula un síndrome

Algoritmo HQC–KEM.KeyGen

Require: Semilla $seed_{KEM}$

Ensure: Clave pública ek_{KEM}, clave privada dk_{KEM}

▷ *Ejecutar HQC–PKE.KeyGen para generar las claves base*

1: $(ek_{PKE}, dk_{PKE}) \leftarrow \text{HQC–PKE.KeyGen}(seed_{KEM})$

▷ *Ampliar la clave privada con la clave pública y el hash de ésta*

2: $hpk \leftarrow H(ek_{PKE})$

3: $dk_{KEM} \leftarrow (dk_{PKE}, ek_{PKE}, hpk)$

▷ *Asignar las claves de salida*

4: $ek_{KEM} \leftarrow ek_{PKE}$

5: **return** (ek_{KEM}, dk_{KEM})

5.3.2.2 Encapsulación

El mensaje es codificado junto con un error, y luego cifrado usando el síndrome público

Algoritmo HQC–KEM.Encaps

Require: Clave pública ek_{KEM}

Ensure: Clave compartida K, texto cifrado $\mathbf{c}_{KEM}$

▷ *Generar una semilla aleatoria y un mensaje temporal*

1: $\mathbf{m} \leftarrow \text{RandomBytes}(K_{\text{len}})$

2: $\theta \leftarrow G(\mathbf{m} \,\|\, H(ek_{KEM}))$ ▷ Derivar aleatoriedad a partir del mensaje y la clave pública

▷ *Cifrar el mensaje con HQC–PKE.Encrypt*

3: $\mathbf{c}_{PKE} \leftarrow \text{HQC–PKE.Encrypt}(ek_{KEM}, \mathbf{m}, \theta)$

▷ *Construir el ciphertext extendido añadiendo el hash del mensaje*

4: $\mathbf{c}_{KEM} \leftarrow (\mathbf{c}_{PKE}, H(\mathbf{m}))$

▷ *Derivar la clave compartida a partir del mensaje y del ciphertext*

5: $K \leftarrow KDF(\mathbf{m} \,\|\, \mathbf{c}_{KEM})$

6: **return** $(K, \mathbf{c}_{KEM})$

5.3.2.3 Desencapsulación

El receptor usa su clave privada para decodificar el error y recuperar la semilla secreta compartida.

Algoritmo HQC–KEM.Decaps

Require: Clave privada dk_{KEM}, texto cifrado $\mathbf{c}_{KEM}$

Ensure: Clave compartida K

▷ *Separar la clave privada en sus componentes*

Parse dk_{KEM} como $(dk_{PKE}, ek_{PKE}, hpk)$

▷ *Separar el texto cifrado*

Parse $\mathbf{c}_{KEM}$ como $(\mathbf{c}_{PKE}, d)$

▷ *Descifrar el mensaje usando HQC–PKE.Decrypt*

$\mathbf{m}' \leftarrow \text{HQC–PKE.Decrypt}(dk_{PKE}, \mathbf{c}_{PKE})$

▷ *Recalcular la aleatoriedad a partir de* $\mathbf{m}'$ *y de la clave pública*

$\theta' \leftarrow G(\mathbf{m}' \,\|\, hpk)$

▷ *Recalcular el ciphertext y verificar coherencia*

$\mathbf{c}'_{PKE} \leftarrow \text{HQC–PKE.Encrypt}(ek_{PKE}, \mathbf{m}', \theta')$

$\mathbf{c}'_{KEM} \leftarrow (\mathbf{c}'_{PKE}, H(\mathbf{m}'))$

▷ *Verificar si el ciphertext recibido es válido*

if $\mathbf{c}'_{KEM} = \mathbf{c}_{KEM}$ **then**

▷ *El descifrado fue correcto, usar la clave derivada de* $\mathbf{m}'$

    $K \leftarrow KDF(\mathbf{m}' \,\|\, \mathbf{c}_{KEM})$

else

▷ *El descifrado falló, usar clave aleatoria oculta*

    $\mathbf{m}'' \leftarrow \text{RandomBytes}(K_{\text{len}})$

    $K \leftarrow KDF(\mathbf{m}'' \,\|\, \mathbf{c}_{KEM})$

end if

▷ *Devolver la clave compartida*

return K

Su diseño favorece la implementación eficiente en hardware gracias a su estructura cuasi-cíclica, es decir, que se comporta de manera similar a un código cíclico, pero con una restricciones adicionales, permitiendo reutilización de recursos.

5.4 Estado del Arte en Implementación de HQC en Hardware

Varios trabajos recientes han explorado el diseño de aceleradores para HQC, estos trabajos se trataron en el apartado de antecedentes.

Sin embargo, a nivel de Colombia no se registran antecedentes académicos o técnicos en

repositorios públicos que documenten la implementación de HQC en hardware. Esto revela un vacío de conocimiento y la oportunidad de realizar aportes locales en el campo de la criptografía postcuántica desde la ingeniería electrónica.

6. METODOLOGÍA

La metodología adoptada para el desarrollo de este anteproyecto se estructura en cuatro fases principales, intencionalmente alineadas con los objetivos específicos que se plantearon. Estas fases permiten abordar el proceso del desarrollo del algoritmo en hardware, desde la comprensión funcional de HQC hasta su validación en una plataforma FPGA, garantizando una implementación verificable y documentable que pueda compararse con desarrollos previos.

ACTIVIDADES

1. **Comprensión y modelado del algoritmo HQC**

   1.1. Analizar la versión más actual del documento oficial para identificar el flujo general del algoritmo, estructura de datos, codificadores y operaciones criptográficas involucradas.

   1.2. Descomponer el algoritmo HQC en etapas clave: generación de claves, codificación, encapsulación y desencapsulación.

   1.3. Implementar cada uno de los bloques del algoritmo HQC en Python, respetando las operaciones indicadas en la especificación.

   1.4. Ejecutar el modelo funcional con datos de prueba, observando el comportamiento de cada módulo para validar la correcta ejecución del modelo funcional comparando sus resultados con los vectores de prueba provistos por el NIST (test vectors oficiales).

   1.5. Documentar el comportamiento esperado de cada módulo y las interfaces entre bloques para su posterior traducción al diseño hardware.

2. **Identificación de bloques funcionales del sistema de encriptación HQC**

   2.1. Estudiar la secuencia de procesos criptográficos del algoritmo para identificar la interacción entre módulos y el orden lógico de ejecución.

2.2. Representar gráficamente el flujo de datos entre los bloques, identificando buses, registros intermedios y rutas de comunicación.

2.3. Proponer un modelo de control basado en una máquina de estados finita FSM que estructure la activación secuencial y paralela de los bloques funcionales.

2.4. Verificar que el diseño del datapath y controlpath permita una arquitectura escalable, reutilizable y adecuada para su futura implementación en VHDL.

2.5. Consolidar la información en diagramas estructurales y tablas descriptivas a modo de documentación, sirviendo como base para el desarrollo RTL en la siguiente fase.

3. **Síntesis del diseño VHDL del algoritmo en FPGA**

3.1. Utilizar la placa de desarrollo DE2-115 con FPGA Cyclone IV de Intel, configurando el proyecto en el entorno Quartus Prime, incluyendo la configuración del dispositivo, asignación de pines, y definición de la arquitectura general del sistema.

3.2. Codificar en VHDL cada uno de los bloques identificados previamente (generador de claves, codificadores RS y RM, multiplicadores, encapsulador y descapsulador), garantizando modularidad y compatibilidad entre interfaces.

3.3. Incorporar estrategias como la reutilización de bloques funcionales comunes, paralelización de operaciones críticas, por ejemplo, en multiplicadores polinómicos y la reducción del consumo dinámico mediante técnicas como el *clock gating*.

3.4. Ejecutar el proceso de síntesis para traducir el diseño lógico en una estructura implementable en hardware.

3.5. Verificación mediante SignalTap (Intel/Altera) para observar en tiempo real el comportamiento del algoritmo durante la ejecución de pruebas permitiendo documentar la correcta secuenciación de señales internas en condiciones operativas reales.

4. **Evaluación de desempeño y comparación con diseños previos**

4.1. Obtener indicadores como el tiempo de ejecución en la encapsulación y desencapsulación, el consumo de recursos lógicos (LUTs, flip-flops, BRAMs), el consumo de potencia estimado mediante PowerPlay power analizer de Intel Quartus.

4.2. Contrastar los resultados obtenidos con los trabajos analizados en la sección de antecedentes, evaluando mejoras o desventajas en rendimiento, área-tiempo y eficiencia energética.

4.3. Confirmar que el diseño final cumple con los requerimientos funcionales y de seguridad definidos por el NIST.

4.4. Registrar los datos obtenidos en cuadros comparativos y gráficas, acompañados de una discusión técnica que justifique la efectividad y aplicabilidad del diseño propuesto.

7. CRONOGRAMA

El cronograma del proyecto se ha organizado de forma secuencial, siguiendo las fases principales: modelado funcional, diseño estructural, implementación en FPGA y evaluación de resultados. Las actividades de síntesis e implementación en VHDL requerirán mayor tiempo debido a la complejidad del diseño, la integración de módulos y la necesidad de validación en tiempo real. De igual manera, la etapa inicial de modelado en Python demanda atención especial, ya que sienta las bases funcionales del algoritmo. EL ultimo objetivo, que corresponde al análisis estructural y la evaluación comparativa, dependerá del avance de las anteriores y tendrá una carga de trabajo más reducida.

El siguiente esquema resume y estima la duración en semanas de cada objetivo especifico con el fin de mantener un control temporal sobre las actividades a desarrollar y así consolidar todos y cada uno de ellos.

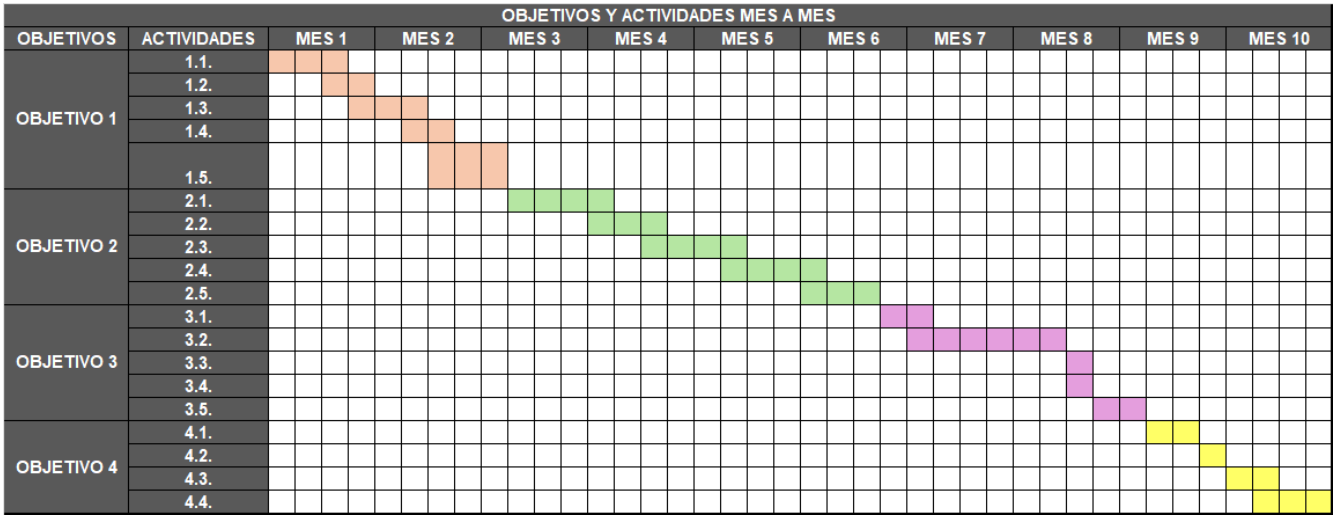


Figura 7.1: Cronograma de actividades

8. PRESUPUESTO

A continuación se presenta el presupuesto estimado para la ejecución del proyecto, considerando los recursos necesarios para llevar a cabo las actividades definidas en el cronograma. Se incluyen costos asociados a recursos humanos, herramientas de desarrollo, software y material bibliográfico. Los criterios para estructurar este presupuesto se realizaron teniendo en cuenta la disponibilidad institucional y necesidades específicas del diseño e implementación del algoritmo en una plataforma FPGA.

RUBRO	DESCRIPCIÓN	UNIDAD	CANTIDAD	VALOR UNITARIO	FUENTES DE FINANCIACIÓN	
					Universisdad del Valle	Tesista
Recursos Humanos	Director - UV	1 h/semana	44	$ 150.000,00	$ 6.600.000,00	-
	Tesista	12 h/semana	480	$ 12.000,00	-	$ 5.760.000,00
Equipos	Computador portatil	-	1	$ 2.500.000,00	-	$ 2.500.000,00
Hardware	Plataforma de desarrollo FPGA	-	1	$ 3.212.000,00	$ 3.212.000,00	-
Material Bibliografico	Base de datos Univalle	-	1	-	-	-
Software y licencias	Entorno de desarrollo Quartus Prime Lite edition	-	1	-	-	-
Imprevistos	-	-	-	-	$ 200.000,00	$ 400.000,00
Subtotal					$ 10.012.000,00	$ 8.660.000,00
Total					$	18.672.000,00

Figura 8.1: Presupuesto general

REFERENCIAS

[1] C. Gidney and M. Ekerå, "How to factor 2048 bit rsa integers in 8 hours using 20 million noisy qubits," *Quantum*, vol. 5, p. 433, 2021.

[2] National Institute of Standards and Technology (NIST), "Status Report on the Fourth Round of the NIST Post-Quantum Cryptography Standardization Process," Internal Report (NISTIR) 8545, NIST, March 2025.

[3] IBM, "What is quantum-safe cryptography?." `https://www.ibm.com/think/topics/quantum-safe-cryptography`, September 2024.

[4] S. Singh and E. Sakk, "Implementation and analysis of shor's algorithm to break rsa cryptosystem security," *TechRxiv*, January 2024.

[5] A. T. Olutimehin, S. Joseph, O. C. Metibemu, A. Y. Balogun, O. O. Olaniyi, and A. O. Olayemi, "Future-proofing data: Assessing the feasibility of post-quantum cryptographic algorithms to mitigate 'harvest now, decrypt later' attacks," *Archives of Current Research International*, vol. 25, p. 1098, February 2025.

[6] Cybersecurity and Infrastructure Security Agency (CISA), "Prepare for a new cryptographic standard to protect against future quantum-based threats." CISA, July 2022.

[7] Y. Laigle-Chapuy, O. Blazy, P. Gaborit, A. Hauteville, and J.-P. Tillich, "Hqc: Ind-cca2 kem based on quasi-cyclic codes, specification v1.1," Tech. Rep. NIST PQC Submission Document, National Institute of Standards and Technology (NIST), February 2025.

[8] National Institute of Standards and Technology, "Nist selects hqc as fifth algorithm for post-quantum encryption." NIST News, March 2025.

[9] X. Wang, S. Xu, P. Liu, and S. Xu, "A high-efficiency hardware design for the post-quantum kem hqc," *IEEE Transactions on Circuits and Systems I: Regular Papers*, 2024.

[10] H. Li, J. Fan, H. Jia, H. Lu, and H. Liu, "An efficient hardware design for fast implementation of hqc," in *2023 24th International Symposium on Quality Electronic Design (ISQED)*, pp. 1–6, IEEE, 2023.

[11] W. Tu, R. Li, F. Zhang, G. Ma, and H. Jia, "Lightweight and time-efficient hardware encoder for post-quantum scheme hqc," *Electronics*, vol. 13, no. 8, p. 1408, 2024.

[12] W. Tu, R. Li, F. Zhang, G. Ma, and H. Jia, "An efficient and unified rtl accelerator design for hqc-128, hqc-192, and hqc-256," *Electronics*, vol. 12, no. 13, p. 2809, 2023.

[13] National Institute of Standards and Technology, "Selected algorithms for the nist post-quantum cryptography standardization project." NIST CSRC, 2025.